Nama: Jonathan Teguh Samuel Kaeng

KONTROL MOTOR DC MENGGUNAKAN ANTARMUKA GUI PYTHON DAN KOMUNIKASI SERIAL

A. Tujuan Modul

Tahap Progress 1 bertujuan untuk merancang dan mengimplementasikan sistem kendali aktuator motor DC berbasis antarmuka grafis (*Graphical User Interface*/GUI) menggunakan bahasa pemrograman Python. Pengujian ini difokuskan pada keandalan transmisi data dua arah melalui protokol komunikasi Serial (via kabel USB) antara antarmuka aplikasi di komputer dengan mikrokontroler ESP32, untuk mengendalikan arah putaran serta modulasi lebar pulsa (PWM) secara *real-time*.

B. Skema Rangkaian

Skema pengkabelan perangkat keras (hardware) ESP32 terhubung dengan driver motor L298N untuk mengendalikan aktuator mekanis, dengan konfigurasi pin sebagai berikut:

- Pin 27 (motor1Pin1): Dihubungkan ke pin IN1 *driver* motor.
- Pin 26 (motor1Pin2): Dihubungkan ke pin IN2 *driver* motor.
- Pin 12 (enable1Pin): Dihubungkan ke pin ENA *driver* motor (sebagai sumber sinyal PWM).

C. Konsep & Logika Sistem

Arsitektur sistem ini mengadopsi model komunikasi *Client-Server* sederhana. Komputer (berbasis Python) bertindak sebagai pengirim perintah (*transmitter*) melalui pustaka pyserial, sedangkan ESP32 bertindak sebagai penerima (receiver). Aplikasi Python akan mengirimkan string perintah spesifik seperti "1" untuk menyalakan motor, "0" untuk mematikan motor, dan "PWM:xxx" untuk mengatur kecepatan. Mikrokontroler senantiasa mendengarkan *port* serial menggunakan fungsi Serial.available() dan mengurai (parsing) instruksi tersebut untuk dieksekusi menjadi sinyal elektris keluaran (HIGH/LOW/PWM).

D. Source Code

a. Source Code Python (Tkinter GUI)

```
import tkinter as tk
from tkinter import ttk, messagebox
```

```
import serial
import serial.tools.list_ports

class MotorControllerApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Python ESP32 Motor Controller")
        self.root.geometry("400x380")
        self.root.resizable(False, False)

        self.serial_port = None

        self.create_widgets()
        self.refresh_ports()

    def create_widgets(self):
        # Frame Koneksi Serial
        conn_frame = ttk.LabelFrame(self.root, text="Pengaturan Koneksi Serial")
        conn_frame.pack(pady=10, padx=10, fill="x")

        self.port_var = tk.StringVar()
        self.port_cb = ttk.Combobox(conn_frame, textvariable=self.port_var,
state="readonly", width=15)
        self.port_cb.grid(row=0, column=0, padx=10, pady=10)

        btn_refresh = ttk.Button(conn_frame, text="Refresh Port",
command=self.refresh_ports)
        btn_refresh.grid(row=0, column=1, padx=5, pady=10)

        self.btn_connect = ttk.Button(conn_frame, text="Connect",
command=self.toggle_connection)
        self.btn_connect.grid(row=0, column=2, padx=5, pady=10)

        # Frame Kontrol Motor Dasar (ON/OFF)
        ctrl_frame = ttk.LabelFrame(self.root, text="Kontrol Motor Dasar")
        ctrl_frame.pack(pady=10, padx=10, fill="x")

        btn_on = tk.Button(ctrl_frame, text="MOTOR ON", bg="#4CAF50",
fg="white", font=("Arial", 10, "bold"), command=lambda:
self.send_command("1"))
        btn_on.pack(side="left", expand=True, fill="x", padx=10, pady=15)

        btn_off = tk.Button(ctrl_frame, text="MOTOR OFF", bg="#F44336",
fg="white", font=("Arial", 10, "bold"), command=lambda:
self.send_command("0"))
        btn_off.pack(side="right", expand=True, fill="x", padx=10, pady=15)

        # Frame Kecepatan (PWM)
        pwm_frame = ttk.LabelFrame(self.root, text="Kontrol Kecepatan (PWM)")
```

```
        pwm_frame.pack(pady=10, padx=10, fill="x")

        self.pwm_slider = tk.Scale(pwm_frame, from_=0, to=255,
orient="horizontal", tickinterval=51)
        self.pwm_slider.set(150) # Set nilai awal ke 150
        self.pwm_slider.pack(fill="x", padx=10, pady=5)

        btn_set_pwm = ttk.Button(pwm_frame, text="Set Kecepatan",
command=self.set_pwm)
        btn_set_pwm.pack(pady=10)

        # Status Label di bagian bawah
        self.status_label = ttk.Label(self.root, text="Status: Disconnected",
foreground="red", font=("Arial", 10, "bold"))
        self.status_label.pack(side="bottom", pady=10)

    def refresh_ports(self):
        """Mencari semua Port COM (Kabel USB) yang tersedia di laptop"""
        ports = serial.tools.list_ports.comports()
        self.port_cb['values'] = [port.device for port in ports]
        if self.port_cb['values']:
            self.port_cb.current(0)

    def toggle_connection(self):
        """Menghubungkan atau memutus koneksi Serial dengan ESP32"""
        if self.serial_port and self.serial_port.is_open:
            self.serial_port.close()
            self.btn_connect.config(text="Connect")
            self.status_label.config(text="Status: Disconnected", foreground="red")
            self.port_cb.config(state="readonly")
        else:
            port = self.port_var.get()
            if port:
                try:
                    # Baudrate 115200 harus sama dengan Serial.begin di ESP32
                    self.serial_port = serial.Serial(port, 115200, timeout=1)
                    self.btn_connect.config(text="Disconnect")
                    self.status_label.config(text=f"Status: Connected to {port}",
foreground="green")
                    self.port_cb.config(state="disabled")
                except Exception as e:
                    messagebox.showerror("Error", f"Gagal terhubung ke {port}:
{str(e)}\n\nPastikan Serial Monitor di Arduino IDE sudah ditutup!")
            else:
                messagebox.showwarning("Peringatan", "Pilih port COM terlebih
dahulu!")

    def send_command(self, cmd):
        """Mengirimkan teks perintah ke ESP32"""
```

```
        if self.serial_port and self.serial_port.is_open:
            try:
                # Mengirim data dengan tambahan newline (\n) agar bisa dibaca oleh
Serial.readStringUntil
                self.serial_port.write((cmd + '\n').encode('utf-8'))
            except Exception as e:
                messagebox.showerror("Error", f"Gagal mengirim data: {str(e)}")
        else:
            messagebox.showwarning("Peringatan", "Silakan klik tombol 'Connect'
terlebih dahulu!")

    def set_pwm(self):
        """Mengambil nilai slider dan mengirimkannya ke ESP32 dengan format
PWM:xxx"""
        val = self.pwm_slider.get()
        self.send_command(f"PWM:{val}")

if __name__ == "__main__":
    root = tk.Tk()
    app = MotorControllerApp(root)
    root.mainloop()
```

b. Source Code C++ (ESP32 Receiver)

```
// KODE ESP32 UNTUK MENERIMA PERINTAH DARI PYTHON GUI
int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

const int freq = 30000;
const int resolution = 8;

void setup() {
  Serial.begin(115200);
  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  ledcAttach(enable1Pin, freq, resolution);

  // Pastikan motor mati saat awal menyala
  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(enable1Pin, 0);

  Serial.println("Sistem Siap! Menunggu perintah dari Python GUI...");
}
```

```
void loop() {
  // Mengecek apakah ada data yang masuk dari kabel USB (Python)
  if (Serial.available() > 0) {
    String command = Serial.readStringUntil('\n');
    command.trim(); // Menghapus spasi atau karakter enter berlebih

    if (command == "1") {
      digitalWrite(motor1Pin1, HIGH);
      digitalWrite(motor1Pin2, LOW);
      ledcWrite(enable1Pin, 200); // Kecepatan default saat ON
      Serial.println(">>> Motor Dinyalakan");

    } else if (command == "0") {
      digitalWrite(motor1Pin1, LOW);
      digitalWrite(motor1Pin2, LOW);
      ledcWrite(enable1Pin, 0);
      Serial.println(">>> Motor Dimatikan");

    } else if (command.startsWith("PWM:")) {
      String pwmValueStr = command.substring(4); // Mengambil angka
setelah teks "PWM:"
      int pwmValue = pwmValueStr.toInt();
      ledcWrite(enable1Pin, pwmValue);
      Serial.print(">>> Kecepatan (PWM) diubah menjadi: ");
      Serial.println(pwmValue);
    }
  }
}
```

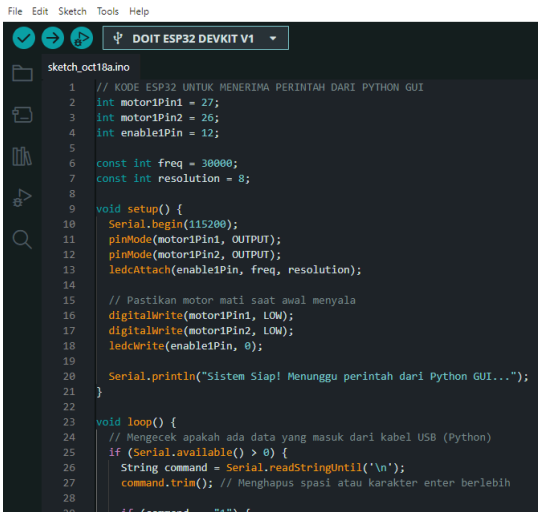
Program terbagi menjadi dua pilar utama: *Front-end* (Python) dan *Back-end* (C++). Pada sisi Python, antarmuka grafis dibangun menggunakan pustaka tkinter, menyediakan kontrol tombol statis dan slider dinamis. Ketika pengguna menekan tombol, fungsi send_command() mengeksekusi pengiriman teks dengan newline (\n) ke COM Port yang aktif. Pada sisi C++ di ESP32, metode Serial.readStringUntil('\n') memegang peran esensial untuk menangkap string perintah tersebut secara utuh, menghapus karakter tak kasat mata melalui metode trim(), lalu mendistribusikannya ke dalam blok kondisi logika (if-else) untuk merespons status aktuator secara presisi.

E. Hasil Pengujian & Analisis

a. Proses Kompilasi dan Koneksi Serial

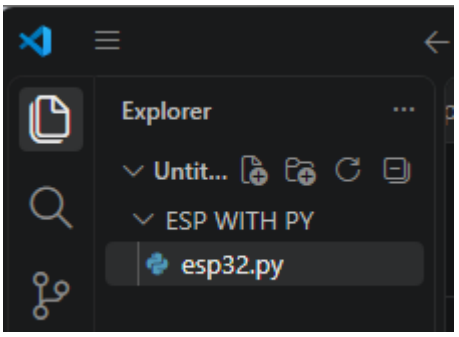
Sebelum pengujian GUI Python dapat dieksekusi, perangkat lunak (*firmware*) berbasis C++ harus dikompilasi dan diunggah ke dalam mikrokontroler ESP32 melalui Arduino IDE. Setelah mikrokontroler

diprogram, langkah selanjutnya adalah mengeksekusi skrip antarmuka Python melalui *code editor* Visual Studio Code (VS Code).



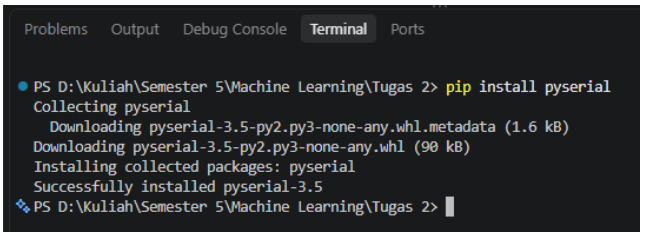
Gambar 1. Proses unggah *firmware* C++ untuk komunikasi Serial. .

Setelah pengunggahan berhasil, langkah selanjutnya adalah menyiapkan lingkungan pengembangan di Visual Studio Code (VS Code) dengan membuat *folder* proyek baru dan *file* bernama esp32.py.



Gambar 2. Pembuatan *file* Python (esp32.py) di VS Code.

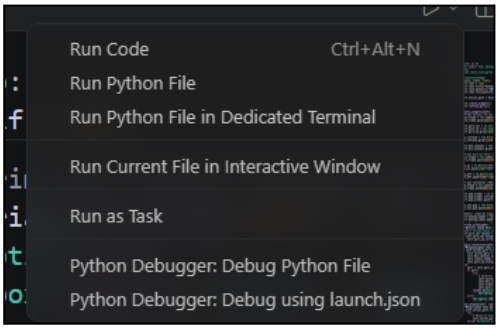
Tahap krusial sebelum peluncuran GUI adalah memastikan terinstalnya modul integrasi *serial port*. Pengguna wajib mengunduh paket dependensi melalui terminal VS Code dengan mengeksekusi perintah pip install pyserial. Pustaka (modul) ini berfungsi sebagai jembatan protokol komunikasi yang memungkinkan ekosistem Python untuk membaca dan menulis data biner/teks langsung ke perangkat keras fisik (ESP32) yang terhubung di *port* USB komputer.



Gambar 3. Proses instalasi dependensi komunikasi data pyserial via terminal.

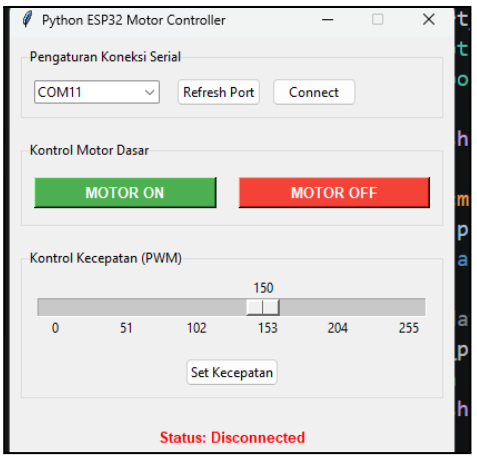
b. Pengujian Respon Aktuator Motor

Setelah seluruh dependensi siap, skrip Python dieksekusi dengan menekan tombol perintah *Run Python File* pada VS Code.



Gambar 4. Eksekusi peluncuran antarmuka GUI Python.

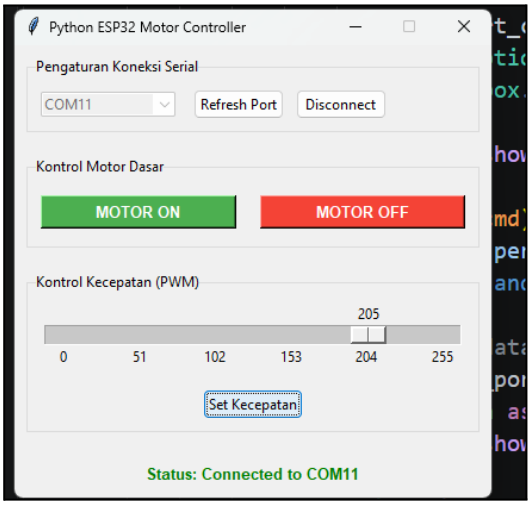
Apabila kompilasi berhasil, sebuah jendela aplikasi *Graphical User Interface* (GUI) berbasis Tkinter akan muncul di layar (seperti pada Gambar 5). GUI ini bertindak sebagai panel kontrol virtual yang terdiri atas elemen pendeteksi *port* otomatis (*Combobox*), indikator status koneksi, tombol instruksi *boolean* (ON/OFF), serta instrumen penyesuaian nilai (*slider*) beresolusi 8-bit untuk memanipulasi rentang parameter PWM.



Gambar 5. Antarmuka panel kontrol virtual motor DC dalam kondisi belum terhubung.

Untuk membangun jembatan transmisi, pengguna harus mengklik tombol *Connect* sambil memastikan kabel USB penghubung antara ESP32 dan komputer tidak terputus. Setelah status koneksi berubah menjadi *Connected*, mikrokontroler siap menerima injeksi data.

Uji coba dilakukan dengan mengeklik tombol "MOTOR ON" / "MOTOR OFF" dan menggeser parameter *slider* PWM. Perlu dicatat bahwa instruksi nilai kelajuan (*slider*) tidak akan ditransmisikan sebelum tombol validasi "Set Kecepatan" ditekan. Hal ini dirancang untuk mencegah terjadinya transmisi data serial (pembacaan kelajuan) yang berlebihan secara konstan yang berpotensi membebani siklus pembacaan di ESP32.



Gambar 6. Kondisi transmisi sinkron ketika GUI terkoneksi ke COM11.

c. Analisis Keseluruhan

Analisis Keseluruhan Berdasarkan serangkaian uji coba yang dilakukan, perancangan antarmuka kontrol motor menggunakan ekosistem Python (Tkinter) terbukti beroperasi dengan tingkat reliabilitas transmisi yang sangat presisi. Arsitektur protokol jabat tangan (*handshake*) serial yang dijembatani oleh pustaka pyserial mampu memfasilitasi komunikasi komunikasi data dua arah antara perangkat lunak tingkat tinggi di komputer dengan eksekutor instruksi tingkat rendah (ESP32) secara instan.

Mekanisme validasi *input* ganda seperti pewajiban penekanan tombol "Set Kecepatan" pasca-penggeseran *slider* menunjukkan sebuah perancangan logika keamanan kontrol yang baik guna mencegah terjadinya fluktuasi (*spamming*) transmisi data beruntun ke memori serial mikrokontroler. Secara keseluruhan, eksperimen tahap awal ini sukses membangun fondasi

pemahaman yang esensial mengenai algoritma *serial parsing*, manajemen status GUI, serta integrasi sinyal aktuator fisik sebelum dikembangkan menuju sistem kendali jarak jauh (IoT) berbasis nirkabel pada tahap (*progress*) selanjutnya.